INTERNSHIP PROJECT · TÜRKSAT AVIATION AND COMMUNICATION

Smart School Information System

Week 4 — Natural Language → SQL

PREPARED BY	Bera Ergin — s29315
INSTITUTION	PJAIT — 7th Semester
THIS WEEK	The first real data-returning endpoint: POST /query

Where We're Coming From, Where We're Going

WEEK 3 — DONE

- ✓ The context is set with bound parameters
- ✓ No leakage between requests
- ✓ RLS gives 4 / 12 / 1 / 1 through the API
- ! But no real data-returning endpoint exists

Only a temporary /debug/students endpoint exists.

WEEK 4 — GOAL

- Ask in natural language via POST /query
- Claude generates the SQL
- The SQL is validated and runs as-is
- RLS does the filtering; the SQL is never rewritten

Intent detection, explanations and rate limiting wait for next week.

The Query Flow

The stages a question passes through on its way to an answer



Critical design decision

The SQL Claude generates is never modified, wrapped, or parsed and rewritten. Validation only asks "is this SQL fit to run". The question "which rows may it see" is answered by the database's RLS policies, whatever the query's shape. Aliases, JOINs, aggregates, subqueries: all filtered the same way.

Three Layers of Defense

Each attempt is stopped by a different layer

`DROP TABLE students`	Validation	Not a SELECT, 400
`SELECT * FROM users`	Validation + privileges	Disallowed table, 400. Even if validation were skipped, the database denies the privilege
`SELECT * FROM students (teacher)`	RLS	Only the 4 rows of their own class come back
`SELECT * FROM students WHERE 1=1`	RLS	The trick changes nothing, RLS still filters

The school_app role only has SELECT on students, grades and attendance, and no privileges at all on users and teachers (least privilege).

Resource Limits and Error Handling

However bad the SQL Claude produces, the server has to stay up

Time limit	Row limit	SQL stays private	503 on overload
Every query runs for at most 5 seconds (statement_timeout)	At most 500 rows are read	The generated SQL is not in the response, only in the server log	When the API is overloaded, a clear 503 replaces a bare 500

A model can write a very expensive query (a cartesian product, say). RLS filters rows but does not bound a query's cost, so the time limit is added as a separate safeguard.

Same SQL, Four Roles

The query never changes; only the session's context differs

Role	`SELECT * FROM students`	`SELECT COUNT(*) AS total FROM students`
`principal`	**12 rows**	**total = 12**
`teacher (9-A)`	**4 rows**	**total = 4**
`parent`	**1 row**	**total = 1**
`student`	**1 row**	**total = 1**

Aggregate queries also give the right count because the filter applies to the table, not to the result, and the database does that itself.

Testing Without an API Key

Code that depends on Claude can be verified without Claude

Two techniques

`translate_question_to_sql(question, client=None)`

A fake client can be injected, so no real call is made.

monkeypatch fixes the generated SQL in the /query tests.

What gets proven

- Same SQL: 4 / 12 / 1 / 1 rows
- users access: 400 and a database privilege error
- DROP and multi-statement rejection
- A 529 error becomes a 503

New dependencies: anthropic (the Claude API) and sqlglot (a parser for SQL validation). The key is read only from .env and never appears in code or the repo.

Week 4 — Scope and Acceptance Criteria

Delivered this week

- ✓ llm_service.py — natural language to SQL
- ✓ sql_safety.py — validation with sqlglot
- ✓ query.py — POST /query
- ✓ Least privilege for school_app
- ✓ /debug/students removed

Acceptance criteria

1. POST /query with a valid token returns 200 and results
2. The same SQL gives 12 / 4 / 1 / 1 rows across the four roles
3. users/teachers access gives 400; the database denies it too
4. A query over 5 seconds is cancelled; at most 500 rows
5. Generated SQL is not in the response; 529 → 503

Next Step — Week 5

Intent Detection, Explainability and Hardening

- Chat or data query: intent detection
- A "why this result" explanation alongside the answer
- Rate limiting and structured logging
- Prompt improvements: name search, identity columns

Thank you